

Generatore di Maschere per Metashape

Rimozione automatica dello sfondo per fotogrammetria di reperti — guida completa di installazione e uso su Windows 11

Questo strumento genera in batch, con accelerazione GPU, le maschere di sfondo per le tue foto JPEG e le salva come PNG pronti per l'import in Agisoft Metashape. Per ogni foto produce una maschera in cui il **bianco** indica l'oggetto da ricostruire e il **nero** lo sfondo da escludere. È pensato per oggetti fotografati senza fondale neutro, dove i metodi interni di Metashape non bastano.

1. Requisiti

- Windows 11 con una GPU NVIDIA (es. RTX 4060) e driver aggiornati.
- **Python 3.10 o 3.11** a 64 bit (consigliato per la massima compatibilità con onnxruntime-gpu e rembg).
- Connessione a Internet al primo avvio (per scaricare librerie e modello).
- Le tue foto in formato JPEG in un'unica cartella.

Non serve installare manualmente il CUDA Toolkit né cuDNN: su Windows lo script li scarica come pacchetti pip (cuDNN 9 e runtime CUDA 12) e li configura da solo. Bastano i driver NVIDIA aggiornati.

Perché non il Python di OSGeo4W/QGIS: quel Python è dedicato a QGIS e potrebbe non includere *tkinter* né gestire bene i pacchetti GPU. Per questo strumento conviene un'installazione Python separata da python.org.

2. Installare Python

Se hai già Python 3.10/3.11, salta al punto 3.

- Scarica l'installer da python.org (sezione Downloads → Windows).
- Avvia l'installer e, **molto importante**, spunta la casella «Add python.exe to PATH» in basso prima di procedere.
- Scegli «Install Now» e attendi la fine.

Verifica dell'installazione — apri il **Prompt dei comandi** (cerca «cmd» nel menu Start) e digita:

```
python --version
```

Dovresti vedere qualcosa come Python 3.11.x. Se compare un errore, Python non è nel PATH: reinstallalo ricordando di spuntare la casella del PATH.

3. Preparare i file

- Crea una cartella di lavoro, ad esempio C:\Maschere.
- Copia al suo interno lo script `genera_maschere_metashape.py`.
- Tieni le foto in una cartella separata, ad esempio C:\foto\conci.

4. Primo avvio e installazione automatica delle dipendenze

Apri il Prompt dei comandi nella cartella dello script. Un modo rapido: apri la cartella in Esplora File, clicca sulla barra degli indirizzi, scrivi **cmd** e premi Invio.

Poi lancia il comando base (sostituendo i percorsi con i tuoi):

```
python genera_maschere_metashape.py --input "C:\foto\conci" --output  
"C:\foto\conci\masks" --downscale 2000
```

Al primo avvio lo script **controlla le dipendenze** (rembg, onnxruntime-gpu, pillow, numpy). Se mancano, te lo segnala e chiede conferma con [S/n]: premi Invio o **S** per installarle automaticamente via pip. Le opzionali (tqdm, la barra di avanzamento) vengono offerte a parte.

Per installare senza alcuna domanda (utile in automazioni), aggiungi `--yes` al comando. Il primo download del modello rembg e delle librerie GPU può pesare alcune centinaia di MB.

GPU NVIDIA: su Windows lo script installa in automatico anche cuDNN 9 e il runtime CUDA 12 (nvidia-cudnn-cu12, nvidia-cuda-runtime-cu12) e ne registra le DLL all'avvio, così onnxruntime trova cudnn64_9.dll e usa la scheda video. Non serve installare il CUDA Toolkit a parte.

5. Scegliere il modello di segmentazione

Se non specifichi `--model`, lo script mostra un menu numerato: leggi pro e contro, digita un numero e premi Invio. Premendo **Invio senza digitare nulla** si usa il modello di default, **ISNet General Use** (id `isnet-general-use`), il più adatto a concetti e reperti. Ecco i modelli disponibili:

Modello	Punti di forza	Limiti	Quando usarlo
ISNet General Use (consigliato)	Bordi netti e precisi; ottimo stacco anche con sfondo poco contrastato	Un po' più lento di U2-Net; ~170 MB	Prima scelta per concetti e reperti
U2-Net	Buon compromesso qualità/velocità; collaudato	Bordi più morbidi; possibili aloni	Riserva se ISNet fallisce
BiRefNet (general)	Massimo dettaglio ai bordi e sugli spigoli	Pesante su VRAM, lento; ~900 MB	Quando serve il massimo dettaglio
U2-Net P	Molto veloce e leggero	Meno preciso, bordi grossolani	Molte foto, priorità alla velocità
U2-Net Human Seg	Ottimizzato per silhouette umane	Inadatto a oggetti inanimati	Da ignorare per l'archeologia

In alternativa puoi forzare un modello da riga di comando, ad esempio `--model isnet-general-use`.

6. Parametri disponibili

Opzione	Valore	Cosa fa
<code>--input</code>	cartella	Cartella con le foto JPEG (obbligatorio).
<code>--output</code>	cartella	Dove salvare le maschere PNG (obbligatorio).
<code>--model</code>	nome	Modello rembg. Se omissso, appare il menu.
<code>--downscale</code>	px (es. 2000)	Lato lungo max per la segmentazione; la maschera torna poi a piena risoluzione. Accelera molto su 24 MP. 0 = piena risoluzione.
<code>--alpha-threshold</code>	0-255 (def. 10)	Sotto la soglia il pixel diventa sfondo. Alza se restano aloni, abbassa se l'oggetto ha buchi.
<code>--feather</code>	px (def. 0)	Sfumatura dei bordi. 0 = bordo netto (consigliato).
<code>--no-matting</code>	—	Disattiva l'alpha matting: bordi più grezzi ma elaborazione più veloce; elimina i PERFORMANCE WARNING di pymatting.
<code>--suffix</code>	testo (def. _mask)	Suffisso del file: IMG_1234 → IMG_1234_mask.png.
<code>--ext</code>	estensioni	Estensioni da processare (es. .jpg .jpeg).
<code>--overwrite</code>	—	Rigenera anche le maschere già esistenti.

--yes / -y

–

Installa le dipendenze mancanti senza chiedere conferma.

7. Esempi d'uso

Comando base con menu modelli e downscale per foto da 24 MP:

```
python genera_maschere_metashape.py --input "C:\foto\conci" --output  
"C:\foto\conci\masks" --downscale 2000
```

Scrivi tutto su **una sola riga**. In PowerShell, se vuoi spezzarlo, usa il backtick ` a fine riga; nel Prompt cmd si usa invece ^.

Modello specifico, bordo netto, senza conferme:

```
python genera_maschere_metashape.py --input "C:\foto\conci" --output  
"C:\foto\conci\masks" --model isnet-general-use --alpha-threshold 12 --yes
```

8. Importare le maschere in Metashape 2.3

In Metashape 2.3 il comando non è sotto *Tools* → *Generate Masks*, ma si chiama **Import Masks** e si raggiunge dal menu contestuale. Dopo aver aggiunto le foto al chunk:

- 1 Seleziona le foto nel pannello **Workspace** o **Photos** (puoi selezionarle tutte; con l'opzione *All cameras* più sotto verranno comunque coperte tutte)
- 2 Clicca con il **tasto destro** su una foto → **Masks** → **Import Masks...** (in alternativa dalla barra menu: **File** → **Import** → **Import Masks...**)
- 3 Nel dialog imposta **Method: From File** (carica le maschere da immagini separate)
- 4 **Operation: Replacement** (sostituisce eventuali maschere preesistenti)
- 5 **Filename template:** {filename}_mask.png (adatta il suffisso se l'hai cambiato)
- 6 **Apply to: All cameras**
- 7 Premuto **OK**, comparirà la finestra di selezione cartella: indica la cartella di output delle maschere (es. C:\Users\efr\Desktop\Maschere\001\A)

Passaggio cruciale per l'allineamento: in *Workflow* → *Align Photos* apri le opzioni avanzate e imposta **Apply masks to** → **Key points** (raccomandato quando tutte le immagini hanno la maschera). Senza questa impostazione lo sfondo continua a generare tie point.

Convenzione di colore — importante: in Metashape l'area **mascherata (nera)** è quella esclusa dal processing. Per escludere lo sfondo bisogna quindi mascherare lo sfondo, non l'oggetto. Lo script genera maschere con **oggetto bianco e sfondo nero**: è la convenzione corretta, lo sfondo viene escluso. Se all'import notassi l'effetto invertito (scompare l'oggetto invece dello sfondo), la tua build interpreta il bianco come maschera: in tal caso vanno invertite.

9. Suggerimenti e risoluzione problemi

Controllo qualità

- Genera prima 3–4 maschere e ispezionale: se l'oggetto ha buchi, abbassa `--alpha-threshold`; se resta sfondo, alzalo o cambia modello.
- Le poche maschere imperfette si ritoccano a mano in Metashape, senza rifare il batch.

Problemi comuni

- **«python» non riconosciuto:** Python non è nel PATH. Reinstallalo spuntando «Add python.exe to PATH».

- **Errore «cudnn64_9.dll is missing»:** manca o non viene trovato cuDNN 9. Lo script ora lo installa e ne registra le DLL da solo; se l'errore persiste, installa a mano con `pip install nvidia-cudnn-cu12 nvidia-cuda-runtime-cu12` e rilancia.
- **CUDA non rilevata / va su CPU:** aggiorna i driver NVIDIA. Funziona comunque su CPU, solo più lentamente.
- **PowerShell e comandi su più righe:** il carattere di continuazione è il backtick (```), non `^` (che vale solo per il Prompt cmd). Meglio tenere il comando su una riga sola.
- **tkinter mancante:** riguarda solo la versione con interfaccia grafica, non quella da riga di comando.
- **«PERFORMANCE WARNING ... Cholesky»:** non è un errore (lo emette pymatting su qualche immagine); la maschera è prodotta lo stesso. Per eliminarlo e velocizzare usa `--no-matting`.
- **Installazione pip fallita:** prova manualmente in un Prompt: `pip install "rembg[gpu]" onnxruntime-gpu pillow numpy`.